



# SIMON

---

## WHITEPAPER

A distributed habitat for open weights. People with GPUs compute inference, and the network verifies they actually did the work.

---

**Version:** 1.0 · 2026-09-17

**Status:** architecture + protocol core; network running on two machines (test), computation verification  
DESIGNED, not implemented

**Concept:** Karol (Szpili) · implementation and measurements: Szpon (OpenClaw) + Claude Code

---

Governing rule: **no evidence, no status** (DESIGNED / IMPLEMENTED / MEASURED).

# 1. Summary

---

SIMON is a **distributed habitat for open model weights**. People with GPUs compute inference, receive a token for it, and the network verifies that they really did the work — without a single server anyone can switch off.

Today a model is a file you download. A file has nowhere to live outside somebody else's API. SIMON is the place where weights **reside**: they run, they are used, they evolve, and access to them flows from the network rather than from one cloud.

**The difference from Hugging Face is the entire point of the design:**

|                        | Hugging Face           | SIMON                  |
|------------------------|------------------------|------------------------|
| Model                  | a file to download     | a process that runs    |
| State                  | frozen at upload time  | alive, used, evolving  |
| Criterion of existence | "somebody uploaded it" | used + healthy + safe  |
| Value                  | download count         | real usage and quality |
| Metaphor               | a library              | a <b>habitat</b>       |

**HF stores. SIMON keeps alive.**

---

# 2. Problem

---

Three things at once:

**2.1. Open weights have no infrastructure.** A publicly released model sits on one server or waits to be downloaded. If the server disappears, the model disappears. There is no mechanism giving it a durable place to run.

**2.2. Access to models is centralised.** Even when weights are open, practical access runs through the APIs of a handful of companies. Whoever owns the API sets the price, the limits and the data-use terms.

**2.3. Consumer hardware sits idle.** Consumer graphics cards are idle most of the day. BOINC and Salad showed that people are willing to donate compute — but not for free, and not without verification of the work.

**What is missing: a mechanism that pays for work genuinely performed and verifiable on somebody else's hardware.**

---

### 3. Thesis

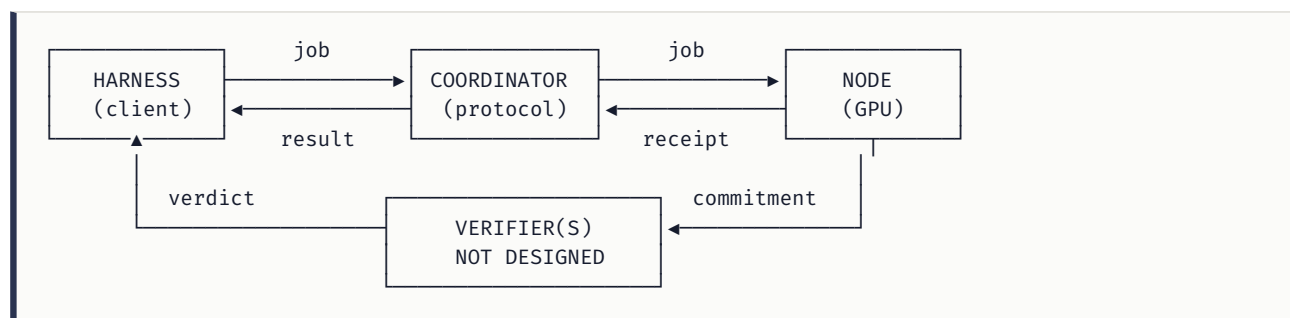
**THINK for verified work; a burned fee as the condition of issuance.**

Three elements, inseparable:

1. **Verification** — the network must be able to check that a node actually computed, rather than merely claiming to.
2. **Lottery** — the client does not pick the node, so it cannot buy "its own".
3. **Settlement** — work pays, cheating loses the stake.

Without verification the rest is pointless: a node that signs a fake result itself passes every check.  
**This is the point where SIMON either stands or falls.**

### 4. Architecture



**Four layers:**

| Layer       | Role                                  | Technology                                  |
|-------------|---------------------------------------|---------------------------------------------|
| Client      | submits work, receives the result     | ACP (JSON-RPC), integrates with any harness |
| Coordinator | assignment, settlement, registry      | Rust                                        |
| Node        | runs the model, assembles the receipt | Rust + vLLM (Python)                        |
| Transport   | announcements, dialling, exchange     | libp2p (gossipsub + request-response)       |

**Flow:**

```
client broadcasts a job (gossipsub – it does not know to whom)
↓
coordinators respond → one wins (deterministic ranking)
↓
a node with a GPU runs the model and assembles a RECEIPT
↓
the client checks the signature and receipt consistency ITSELF (it does not trust the
coordinator)
↓
[VERIFIER] recomputes activations and issues a verdict ← ROLE NOT DESIGNED
```

**A receipt is a signed claim, not a proof.** The node submits a claim of execution together with material for probabilistic auditing. The signature alone confirms the **author of the receipt**, not the correctness of the computation.

**What the client can check itself, and what it cannot.** It verifies the signature, the `job_id` and model match, and — since 2026-09-18 — **that the digest in the receipt covers the text actually received** (previously the signature covered only metadata, so a node could return any text; see

`docs/SECURITY-LOG.md` ).

**CORRECTION.** An earlier version of this paragraph said the client verifies that the receipt is single-use. **No such check exists.** Receipt reuse is prevented by something else: `order_id` is a digest of an order carrying a fresh nonce, so `job_id` is unpredictable and a receipt from another job fails the gate. That is a consequence of construction, not a check — and it gives no replay detection across clients or over time. Accounting needs a ledger (M5.2), because without one nothing notices the same receipt being counted twice.

It **cannot** establish on its own whether the node actually computed — that requires the model weights and the compute to recreate the activations. A client without a GPU either trusts a third party or has no computation verification at all. Hence a separate role:

**VERIFIER — a role that is REQUIRED but NOT `DESIGNED`.** An independent verifier recomputes the indicated activations and issues a signed verdict. Its assignment, randomisation, compensation and collusion resistance are **required but undesigned**. We do not yet write "committee of 3-7" or a quorum — those are design hypotheses, not decisions.

**Architectural decision:** SIMON is an **ACP agent**, not a client. The harness sees a single agent; the swarm is invisible. One adapter → it works with Claude Code, OpenClaw, dsh and any other harness that speaks ACP. (ACP is a Zed standard, Apache-2.0 — not tied to a single vendor.)

---

## 5. Verification — the heart of the project

---

Three layers:

5.1. TopLoc — probabilistic activation audit.

A fingerprint of selected-layer activations gives a **signal** that the declared model processed the given input. The verifier recomputes the indicated activations and compares them with the executor's commitment.

The mechanism **is not a complete cryptographic proof of the whole inference**: its guarantees depend on the controlled layers, the sampling method, numerical tolerance and the adversary model.

Status: DESIGNED at concept level; NOT IMPLEMENTED; NOT ADVERSARIALLY MEASURED.

! **Boundary**: this is not a proof in the SNARK sense. Newer analyses describe attacks that stay **outside the controlled top-k** of the final layer. TopLoc is meant to be a **signal in the protocol**, not a verdict — hence the need for random, multi-point auditing and an independent verifier role.

! **This is not our invention — it is the state of the art**. The method is published (arXiv:2501.16007, January 2025); Prime Intellect used it to verify rollouts in INTELLECT-2 (arXiv:2505.07291 — distributed 32B training on other people's hardware). **Our work is the network around it** (settlement, node selection, penalties, economics), not the method itself.

**5.2. VRF (job lottery)**. The client does not pick the node, so it cannot buy "its own". The draw happens per session, not per turn (justified in §6).

**5.3. Reputation + penalty**. Stake at risk; a false result means losing it.

**The honest state of affairs**: today the code verifies **only the signature**, not the activations.

`activation_hash` is a plain string — **nobody recomputes it**. A node that signs a fake hash and arbitrary text passes the entire verification. Today a receipt proves "registered node X claims so", not "it computed".

**The real moment of truth**: Frank computes the model, Szpon verifies, and the cheater gets caught.

## 6. Eavesdropping — a boundary that must be stated

Two different attacks; they must not be conflated:

| Attack                                                  | Does TopLoc catch it                                      |
|---------------------------------------------------------|-----------------------------------------------------------|
| A node pretends to be model X and proxies to Claude/GPT | <b>YES</b> — activation hashes will not match the weights |
| A farm collects prompts for distillation                | <b>NO</b> — the node computes honestly, it merely records |

**The key number (ESTIMATE, not measured)**:

| Eavesdropper's share | Draw per turn, 50 turns | One node per session |
|----------------------|-------------------------|----------------------|
| 1%                   | 39.5%                   | 1%                   |
| 5%                   | 92.3%                   | 5%                   |
| 10%                  | 99.5%                   | 10%                  |

**Where the difference comes from:** agent harnesses send **the entire context on every turn** (files, history, code). So drawing per turn achieves nothing — a single turn already contains everything. **The node must be drawn ONCE PER SESSION.**

**Three uncomfortable conclusions:**

1. **Corporations and states own the most GPUs.** A lab with 10% of the capacity collects real programming tasks — the most valuable training data there is. **And SIMON pays it in THINK for the privilege.**
2. **The MVP public network protects code WORSE than a commercial API.** An API at least has contractual data-use terms; an anonymous node has none. The slogan "don't feed the corporations" is, in this version, **untrue.**
3. **This cannot be fixed with cryptography on consumer hardware.** FHE  $\approx 3$  s/token; confidential GPU computation (TEE) exists only on server-grade cards — that is, on corporate hardware.

**What actually works (strongest first):**

1. **Private pods** — the user selects "only nodes from my list" (their own, friends', the company's). **The only real protection on consumer hardware.** For code it should be the DEFAULT.
2. **Client-side anonymisation** before sending — limitation: the logic of the code is still visible.
3. **Canaries per node** — a leak identifies the culprit and justifies slashing the stake. Works only for public leaks.
4. **An explicit label:** "public network = treat your prompt as public".

**A line of enquiry investigated and closed (so nobody returns to it):** "a key applied to the model" (client-side encryption of weights/activations). Tested experimentally — **it does not work:**

"You can encrypt a pipe; you cannot encrypt a computer that computes on your behalf."

A key on the weights encrypts nothing (permuting neurons does not change the function). A key on activations breaks against ReLU and LayerNorm. Even where the mask survives, **the model is public**, so the node recovers the key by dividing out the weights. There is no third path: either FHE/MPC, or TEE.

## 7. Economics

**What is new in SIMON is inference verification, not token economics.** Token economics is the best-documented minefield in crypto — other people's mistakes are written up and can be avoided.

**Ninth pitfall — wash-compute (added 2026-09-17 after external audit):**

The audit proposed a remedy: "emission  $\leq$  a share of net burned fees". **This does not remove the mechanism — it only shifts the threshold.** A farm that needs compute anyway still runs a positive balance: it gets the work *and* a share of emission. For an AI farm, wash-compute is not fraud but **optimization** — I compute locally instead of paying someone else. The emission threshold sets the size of the bonus; it does not close the path.

The balance of a party controlling both client and executor: **profit = emission – irreversible fee – electricity – verification cost.** Part of the executor payment returns to the same pocket. If emission exceeds the irreversible costs, the self-transaction remains profitable.

**At this stage we know of no permissionless mechanism that distinguishes real demand from an economically equivalent self-transaction. Therefore emission tied to work volume remains BLOCKED.**

**The cleanest MVP model (no per-work emission):** the client pays the executor for the service; the verifier receives a share of that same fee; the protocol takes a small settlement fee; **there is no additional emission for a completed job.** THINK can be a non-transferable settlement credit, and SIMON emission is not launched before simulation demonstrates real, unrelated demand. **This removes wash-compute profitability** — you can still assign work to yourself, but with no protocol subsidy there is nothing to extract.

**Conclusion: wash-compute is unsolvable at the protocol level.** The protocol cannot know whether the payer and the node are the same party — unless identity data is public, which breaks prompt privacy. This is a **fourth boundary** alongside eavesdropping, alignment and activation invertibility — not a gap to close, but a boundary to name.

**Precedent:**

| Project            | What it did                          | Lesson                                                                   |
|--------------------|--------------------------------------|--------------------------------------------------------------------------|
| Golem (2016)       | a market for compute paid in a token | supply existed, <b>demand did not, for years</b>                         |
| Helium             | rewards for hotspots                 | hundreds of thousands of hotspots, pennies in revenue; location spoofing |
| Filecoin           | issuance for disk space              | Filecoin Plus → <b>fictitious self-dealing contracts</b>                 |
| Livepeer           | verified transcoding + penalty       | <b>technically the closest to SIMON</b>                                  |
| Helium DC / Render | <b>burn-and-mint</b>                 | a ready pattern for a "compute voucher"                                  |

**The failure common to all of them: issuance to suppliers grows faster than user demand.** The network grows by hardware rather than by usage — issuance funds farms, not the product.

## Proposed pattern (A PROPOSAL, not a decision):

```
SIMON – scarce token, TRANSFERABLE
  issuance: fixed per epoch, divided by share of PAID,
            VERIFIED work (not by token count!)
  ⇔ wallet to wallet: OK

THINK – voucher, NON-TRANSFERABLE
  created by: BURNING SIMON, at a fixed price per unit of compute
  spent on jobs
  cannot be sold → cannot be speculated on,
  nor can a miner's priority be bought
```

**Why a non-transferable voucher:** the miner has priority over the buyer (the cooperative principle). If THINK could be transferred, a whale would buy THINK from miners **and their priority along with it** — the market would sidestep the rule with a single transfer.

**Why fixed issuance per epoch:** "X THINK per N tokens" means inflation tied to NVIDIA's progress. Fixed issuance per epoch (like difficulty in BTC) gives faster cards a larger share but **does not expand supply**.

**Why a fixed price per unit of compute:** the user must be able to plan a budget, and the miner pays the electricity bill in local currency. Only the scarce token floats.

**Law and tax — gates, not footnotes:** MiCA (a transferable token offered publicly probably triggers a white paper obligation; exchange requires a CASP licence), and personal income tax in Poland (payment in cryptocurrency for a service probably makes every job a taxable event, which kills the UX). **This needs a lawyer, not guesswork.**

**Order of work:** economics **after** verification. As long as the cheater is not caught, issuance would reward cheaters.

## 8. Governance — throttling, not banning

**The problem:** "safe / non-pathological" is not measurable. Somebody has to decide. A central committee is the very leviathan we are running from. The market alone is no answer either — pathology can be popular.

**The resolution:** the network regulates by **DEGREE**, not by verdict.

| Model                 | What happens                              | Reversible? |
|-----------------------|-------------------------------------------|-------------|
| Used, healthy, safe   | full allocation of hosts and compute      | —           |
| Questionable          | fewer hosts / less compute                | <b>YES</b>  |
| Used for attacks/harm | degraded to a minimum ( <b>not zero</b> ) | <b>YES</b>  |

#### Four reasons for gradation:

1. **There is no central verdict** — there is an allocation function, and it runs automatically.
2. **It is reversible** — a model that fixed its problem regains resources. A ban is a sentence for life; degradation is a **state**, not a label.
3. **It matches the principle "usage decides which model lives"** — a bad model is not forbidden; it simply goes unused and receives no resources.
4. **It matches the cooperative principle** — allocation is economic.

**A gap, stated openly:** how does the network know a model is being used for harm? A content classifier **contradicts the prompt-privacy principle** — to detect "harm in the content" you must see the content. **Feasible only through behavioural detection (patterns at the network level, not the content level) and client reports.**

**Honestly:** graduated degradation is a **hypothesis, not a proven solution**. There is no evidence that it actually cuts off pathology — a bad actor can simply use a different model. It requires simulation.

## 9. What is MEASURED (not estimated)

| Component                                       | Value                                                           | Source                                     |
|-------------------------------------------------|-----------------------------------------------------------------|--------------------------------------------|
| TTFT                                            | 1.74 s                                                          | <b>MEASURED</b> (vLLM, prefix caching OFF) |
| Generation, 27B on an RTX 3090                  | 21.8 tok/s                                                      | <b>MEASURED</b>                            |
| Prefill (sessions of 500→36,500 tokens)         | ~2500-2900 tok/s, stable to 99% of the limit                    | <b>MEASURED</b>                            |
| Error of the character heuristic (Polish prose) | <2%                                                             | <b>MEASURED</b>                            |
| Error of the heuristic (dense JSON)             | ~48%                                                            | <b>MEASURED</b>                            |
| Error of the heuristic (base64)                 | ~65%                                                            | <b>MEASURED</b>                            |
| Error of the heuristic as a function of length  | <b>a constant percentage; independent of length</b>             | <b>MEASURED</b>                            |
| RAM spill above the threshold                   | <b>does not occur</b> (the KV pool is pre-allocated at startup) | <b>MEASURED</b> to 99% of the limit        |
| Network (transport)                             | ~12 tok/s                                                       | ! ASSUMPTION, not measured                 |
| TopLoc verification                             | ~1-3 s                                                          | ! estimate from the paper                  |

**What the heuristic measurements tell us:** the error depends on **the type of content, which is not known in advance** (prose/JSON/base64), not on length. **It cannot be patched with a single multiplier**

— the only way out is to ask the tokeniser for the truth ( `/tokenize` ). That closed the question of whether a heuristic would do.

**A 500-token answer: ~25-30 s** — and the user **sees nothing for the whole duration**, because the result is shown only after verification.

! **The biggest thing to reconsider.** "Verify before showing" destroys the UX at 22 tok/s. The DeepSeek API returns the same thing in 2-3 s and streams it. The alternative: **optimistic delivery** (stream immediately, sample-check afterwards, and slash the cheater's stake). **These numbers make it worth revisiting. This is open, not settled.**

## 10. Boundaries — stated openly, because a defence may not stay silent

| Problem                                     | Status                                                                                                                                        |
|---------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| A cheater faking the result                 | ! <b>DESIGNED</b> (TopLoc) — the code <b>does not verify activations</b> , so today it is not caught                                          |
| Eavesdropping at the node                   | ✗ not fixable on consumer hardware (FHE ~3 s/token, TEE server-grade only)                                                                    |
| A node posing as a proxy to Claude/GPT      | ! <b>DESIGNED</b> — TopLoc catches this <b>provided</b> verification works                                                                    |
| Distillation farm (hoarding prompts)        | ✗ undetectable — it computes honestly                                                                                                         |
| Invertibility of activations                | ✗ a prompt can be reconstructed from activations (arXiv:2505.18332) — "the node sees activations, not text" <b>is not a privacy guarantee</b> |
| Semantic alignment (is the model poisoned?) | ✗ OPEN — deliberately unresolved                                                                                                              |
| Sybil (one operator = 1000 nodes)           | ! requires simulation                                                                                                                         |
| Prompt privacy on the public network        | ✗ one node sees the text (MVP decision)                                                                                                       |

**Conclusion for the user:** if your prompt is valuable, use **private pods**. The MVP public network protects code **worse than a commercial API**.

## 11. Supply and demand

**The bottleneck is DEMAND, not supply.** People are happy to mine (BOINC, Salad), but why would anyone **buy** THINK when the DeepSeek API costs  $\approx$  \$0.60 per million tokens and answers ten times faster?

**Real sources of demand:**

1. **Miners spend their own tokens** — the only natural demand from day one
2. Censorship resistance / no single provider
3. Privacy — **but the MVP decision (one node sees the prompt) undermines it.** It cannot be sold as "private".

**The model base — card memory sets the supply** (Q4  $\approx$  0.6 GB per billion parameters):

| Card                           | Model in practice                          |
|--------------------------------|--------------------------------------------|
| 8 GB                           | 7-8B                                       |
| 12 GB                          | $\leq 14B$                                 |
| 24 GB                          | 27-32B dense / MoE $\sim 30B$              |
| 48-96 GB                       | 70B, larger MoE                            |
| $\sim 475$ GiB (DeepSeek V4.1) | out of reach — needs local pods, not a WAN |

**Consequence:** in its first year SIMON sells open models up to  $\sim 30B$ , **not the frontier**. Acceptable for code, summaries and batch processing. **It does not compete with the best agentic models.**

**Stages:**

| Stage               | Active nodes                                                       | Assumption                                |
|---------------------|--------------------------------------------------------------------|-------------------------------------------|
| Closed alpha        | 2-10                                                               | our own machines + friends                |
| Public alpha        | hundreds-thousands of installs, <b>50-300 active after 30 days</b> | the usual post-launch drop-off            |
| Token on the market | thousands quickly, but farms and speculators                       | sybil + law (MiCA, KYC/AML) are the gates |

## 12. Implementation status

**Code:**  $\sim 5800$  lines of Rust (+ tests). Protocol core, transport (libp2p), coordinator, node, harness.

**Done and verified live (2026-09-17):**

| Element                                                                | Evidence                                                          |
|------------------------------------------------------------------------|-------------------------------------------------------------------|
| Context-limit gate based on truth ( <code>/tokenize</code> )           | smoke test: declared 1000 vs 1400 in reality → controlled refusal |
| Chunking + map-reduce for files above a node's limit                   | live: 20/20 chunks, reduction 20→2→1, zero timeouts               |
| Agent→agent chain (CODER→TESTER)                                       | mock + live, both stages with receipt verification                |
| Per-process <code>peer_id</code> (previously identical for every node) | live: two nodes → two different IDs                               |
| Two concurrent agents on two GPUs                                      | live: Szpon/qwen3.8-27b + Franko/Bielik                           |
| Transport: libp2p's default 10 s timeout (a hidden bug)                | found live, raised to 300 s                                       |
| Portability (macOS)                                                    | audit: no native-C dependencies; <code>temp_dir()</code> fix      |

**What is missing:**

| Element                            | Status                                        |
|------------------------------------|-----------------------------------------------|
| Activation verification (TopLoc)   | <b>DESIGNED</b> — this is the moment of truth |
| A ledger with transaction finality | <code>BurnProof</code> is synthetic today     |
| The ACP layer                      | designed, not implemented                     |
| Simulations (sybil, economics)     | not run                                       |
| Model router (sleep mode)          | a plan, not code                              |

**Honestly: 50-80% of today's code will be rewritten before production.** That is normal, and it is the reason to freeze the format only after verification.

### 13. Roadmap

**M0** (holes in the protocol) → **M1** (transport + binary) → **M2** (two nodes, real vLLM) → **M2.5** (TopLoc + a cheater caught) → **M2.7** (format freeze + review) → **M2.9** (10-12 cases) → **M3** (the ACP layer).

**M2.5 is the moment of truth.** Everything before it is preparation; everything after it is expansion.  
**Until a cheater is caught, we do not know whether SIMON works.**

**What we deliberately do NOT do:** federated learning (nobody needs it for an MVP), guards per specialisation (measure first), "context before model" (a control test showed a workaround, not a fix), a coordinator registry as governance (a static file is a deliberate MVP simplification).

## 14. Principles that came out of this work

---

**1. No evidence, no status.** Three statuses instead of "done": - `DESIGNED` — the decision exists - `IMPLEMENTED` — only with a reference to the test that **failed before the fix** - `MEASURED` — only with a reference to the file containing the result

**Reason:** four times in a single session a status was written down faster than it was checked. The roadmap claimed "Computational Integrity DONE" — the code did not implement it.

**2. Review before the format is committed, not after.** A reviewer found 4 holes in code with 50 green tests, then 3 more (a signed `OrderAccepted`, an `order_id` derived from a counter, and the absence of computation verification).

**3. Honesty about boundaries is part of the product.** A document that stays silent about what it cannot do is worse than no document at all — because people make decisions based on it.

---

## 15. Open questions

---

1. **Should the result be shown before verification?** (UX vs verification — unresolved)
  2. **Does SIMON publicly promise "your data does not go to corporations"?** Honestly, that can be promised **only in private pods**.
  3. **Where does the signal "this model is used for harm" come from?** (behaviour / reports — never content)
  4. **How is model degradation measured?** (loss of hosts / queue priority / stake multiplier)
  5. **Does graduated degradation actually cut off pathology, or merely move it to another model?** (requires simulation)
  6. **What is the minted:burned ratio in issuance?** (requires simulation)
  7. **Does a public network make product sense** at all, given that it protects worse than an API? (private pods as the answer)
- 

## 16. In one sentence

---

**SIMON is the place where open weights reside — people with GPUs compute, the network checks that they really did, and a model lives for as long as it is used, healthy and safe.**

---

*Decision record:* `memory/decisions/` (9 architectural files + 70+ D-\* entries). *Roadmap:* `ROADMAP.md`.  
*Code:* `crates/` (Rust, ~5800 lines). *Governing rule:* no evidence, no status.